

PhishingHunterAI - Progetto Serious Game Phishing

Università di Salerno - Corso di Laurea Magistrale in
Informatica

Studenti:

Vincenzo Tarantino Matr. 0522501885
Bartolomeo Mazzeo Matr. 0522501967

Anno Accademico 2025/2026

Indice

1	Introduzione	2
2	Scelta e preprocessing del dataset	3
2.1	Fonti dei Dati	3
2.2	Evoluzione delle Versioni del Dataset	3
2.2.1	Versione 1: Dataset Base	3
2.2.2	Versione 2: Dataset Sanitizzato	3
2.2.3	Versione 3: Dataset Aumentato	4
3	Fine-Tuning e Analisi Sperimentale	5
3.1	Metodologia di Addestramento e Setup Tecnico	5
3.1.1	Strategia di Fine-Tuning: QLoRA	5
3.1.2	Iperparametri Costanti	5
3.1.3	Preprocessing per Instruction Tuning (Conversione JSONL)	6
3.2	Analisi Sperimentale e Valutazione dei Risultati	6
3.2.1	Esperimento 1: Training su Dataset Raw	6
3.2.2	Esperimento 2: Training su Dataset Sanitizzato	6
3.2.3	Esperimento 3: Training su Dataset Aumentato	7
4	Backend – Generazione e-mail e logica di gioco	9
4.1	Generazione Mail	9
4.2	Post-Processing e Sanitizzazione delle E-mail	10
4.3	Reinforcement Learning	10
4.3.1	Versione scartata - Approximate Policy Evaluation	10
4.3.2	Versione implementata – Algoritmo ELO-like	11
5	Motivator	14
5.1	Motivator Mistral	14
5.2	Motivator Fine Tuning – TinyLlama	14
6	Analisi delle Interazioni e Casi d’Uso	16
6.1	Avvio del Gioco e Scelta del Profilo	16
6.2	Analisi degli Scenari	18
6.3	Inserimento della Motivazione e Valutazione	21
6.4	Condizioni di Fine Partita (Win & Game Over)	23
7	Conclusioni e Sviluppi Futuri	25
7.1	Limiti Riscontrati	25
7.2	Prospettive di Miglioramento	25

Capitolo 1

Introduzione

Nel panorama attuale della sicurezza informatica, il fattore umano rappresenta ancora l'anello debole della catena difensiva aziendale. Le statistiche recenti confermano che la maggior parte delle violazioni dei dati inizia con un errore umano, spesso indotto da tecniche di ingegneria sociale sempre più sofisticate.

Il progetto **Phishing Hunter AI** nasce con l'obiettivo di rispondere a questa esigenza, creando un **Serious Game** educativo progettato per allenare i dipendenti aziendali a riconoscere, analizzare e neutralizzare attacchi di phishing in un ambiente sicuro e controllato.

A differenza dei simulatori statici tradizionali, che si limitano a proporre template predefiniti e scenari ripetitivi che l'utente tende a memorizzare meccanicamente, questo sistema introduce un approccio dinamico e adattivo. Il cuore del progetto risiede nell'utilizzo dell'**Intelligenza Artificiale Generativa** (LLM) per creare e-mail uniche, contestuali e verosimili in tempo reale.

Questa capacità generativa è abbinata a un motore di **Reinforcement Learning** che funge da gestore del gioco: il sistema analizza le performance del giocatore e adatta la difficoltà degli scenari al suo livello di competenza. Questo garantisce una curva di apprendimento ottimale, evitando la frustrazione derivante da sfide troppo complesse o la noia causata da compiti banali, massimizzando così l'efficacia formativa del percorso.

Capitolo 2

Scelta e preprocessing del dataset

Nella fase preliminare del progetto, è stata condotta un'analisi delle risorse disponibili in letteratura per individuare dataset affidabili contenenti esempi di e-mail legittime (*legit*) e malevole (*phishing*).

2.1 Fonti dei Dati

La scelta è ricaduta su due dataset standard nel campo della cybersecurity:

- **Enron Corpus:** Utilizzato come fonte primaria per le e-mail legittime. Questo dataset contiene una vasta raccolta di e-mail aziendali reali provenienti dalla Enron Corporation.
- **Jose Nazario's Phishing Corpus:** Utilizzato come fonte per le e-mail malevole, contenente attacchi reali raccolti tramite honeypots e segnalazioni.

Al fine di ottimizzare i tempi di addestramento e ridurre il carico computazionale, si è proceduto inizialmente all'unione e al campionamento di questi due corpus, creando una base di dati gestibile ma rappresentativa.

2.2 Evoluzione delle Versioni del Dataset

A partire da questa base, il dataset è stato iterato ed evoluto in tre versioni distinte:

2.2.1 Versione 1: Dataset Base

Rappresenta l'unione diretta e non filtrata dei campioni estratti dai due corpus originali. Questa versione è servita come *baseline* per valutare le prestazioni iniziali.

2.2.2 Versione 2: Dataset Sanitizzato

È stata effettuata un'operazione di pulizia profonda, concentrandosi in particolare sul corpus Enron. Sebbene classificato come "legit", il dataset originale Enron contiene comunque una percentuale di spam e comunicazioni non pertinenti.

Attraverso tecniche di filtraggio, sono state rimosse queste "impurità" (email di phishing/spam erroneamente etichettate come legit) per garantire che la classe *Legit* fosse il

più pura possibile, riducendo il rumore che avrebbe potuto confondere il modello durante il training.

Tuttavia, questo processo di filtraggio ha comportato una riduzione volumetrica del dataset e l'introduzione di uno sbilanciamento tra le classi.

2.2.3 Versione 3: Dataset Aumentato

Per superare i limiti quantitativi della versione precedente, mitigare lo sbilanciamento delle classi e modernizzare i vettori di attacco (spesso datati nei dataset storici), è stata applicata una strategia di Data Augmentation Generativa.

Utilizzando un Large Language Model, nello specifico Zephyr-7B-beta, sono state generate migliaia di e-mail di phishing sintetiche per aumentare, variare e modernizzare gli esempi real e phishing.

Questo approccio ha permesso di:

- Introdurre vettori di attacco attuali e contestuali, come la *CEO Fraud*, notifiche cloud fake e scenari di *Spear Phishing* mirati, assenti nei dati dei primi anni 2000.
- **Diversificazione Semantica:** Arricchire il training set con varianti linguistiche e stilistiche complesse, migliorando la capacità di generalizzazione del modello finale.

È importante sottolineare la scelta strategica di utilizzare Zephyr per la generazione dei dati, differenziandolo dal modello target del fine-tuning: Mistral-7B. Questa scelta è stata adottata per evitare il rischio di scarsa generalizzazione del modello usato per il finetuning.

Capitolo 3

Fine-Tuning e Analisi Sperimentale

Questo capitolo descrive la metodologia adottata per l'addestramento del Large Language Model (LLM) e analizza l'impatto qualitativo delle diverse versioni del dataset sulle performance generative del modello.

3.1 Metodologia di Addestramento e Setup Tecnico

Per garantire la coerenza sperimentale e isolare l'efficacia dei dati come unica variabile indipendente, è stata mantenuta una configurazione di training uniforme per tutte le prove effettuate.

3.1.1 Strategia di Fine-Tuning: QLoRA

Dato l'elevato costo computazionale richiesto per il *Full Fine-Tuning* di un modello da 7 miliardi di parametri, si è optato per la tecnica **QLoRA (Quantized Low-Rank Adaptation)**.

Questa strategia combina due approcci fondamentali:

- **Quantizzazione a 4-bit (NF4):** Il modello base *Mistral-7B-Instruct-v0.3* è stato caricato con precisione ridotta per minimizzare l'occupazione di memoria VRAM senza degradare significativamente le performance.
- **Adattatori LoRA:** Invece di modificare tutti i pesi della rete, sono stati addestrati solo piccoli adattatori (matrici di rango ridotto) iniettati nei layer di attenzione del modello.
- **Parametri LoRA:** $\text{Rango (r)} = 16$, $\text{Alpha} = 32$, $\text{Dropout} = 0.05$.

3.1.2 Iperparametri Costanti

Per ogni prova effettuata, sono stati utilizzati i seguenti iperparametri, ritenuti ottimali per task di generazione testo su dataset di medie dimensioni:

- **Numero di Epoche:** 2-3 (per evitare overfitting essendo dataset con un numero ridotto di samples).
- **Learning Rate:** $2e-4$
- **Batch Size:** 4

3.1.3 Preprocessing per Instruction Tuning (Conversione JSONL)

Un passaggio cruciale è stato l'adattamento del dataset al formato richiesto per l'*Instruction Tuning*. Le email raw non potevano essere fornite direttamente al modello; è stato necessario incapsularle in una struttura "Domanda-Risposta". Ogni entry del dataset è stata convertita in un oggetto JSONL strutturato secondo il template di Mistral.

3.2 Analisi Sperimentale e Valutazione dei Risultati

A seguito della fase di preparazione dei dati, sono stati condotti tre esperimenti distinti di *Fine-Tuning* per valutare empiricamente l'impatto della qualità del dataset sulle capacità generative del modello. Di seguito si riporta l'analisi dettagliata per ogni configurazione.

3.2.1 Esperimento 1: Training su Dataset Raw

Il primo esperimento, condotto sulla versione non filtrata del dataset, ha prodotto risultati insoddisfacenti. L'analisi qualitativa degli output ha evidenziato che il modello non è riuscito ad apprendere la struttura semantica richiesta (istruzione/email), limitandosi a memorizzare statisticamente le sequenze di parole più frequenti. Le e-mail generate risultavano spesso prive di coerenza sintattica e consistevano in concatenazioni di parole senza un filo logico contestuale. Questo ha confermato che la sola quantità di dati, in presenza di forte rumore, è insufficiente per un *fine tuning* efficace.

3.2.2 Esperimento 2: Training su Dataset Sanitizzato

In questa configurazione, il modello è stato addestrato sul dataset pulito dalle impurità. Data la notevole riduzione del numero di esempi e il conseguente sbilanciamento tra le classi, a favore delle e-mail Legit, è stata introdotta una tecnica di Weighted Loss, per penalizzare maggiormente gli errori sulla classe minoritaria (*Phishing*).

Analisi della Loss: Sebbene il modello abbia mostrato una capacità migliorata nel distinguere i contesti, il grafico della *Training Loss* 3.1 evidenzia problematiche di convergenza:

- **Instabilità:** La curva appare "frastagliata" e rumorosa, indice che il modello faticava a generalizzare correttamente a causa della scarsità di dati eterogenei.
- **Valore Finale Elevato:** La loss si è assestata su valori relativamente alti (tra 4.0 e 6.0), suggerendo che il modello non ha raggiunto un livello di confidenza ottimale nella predizione del testo.

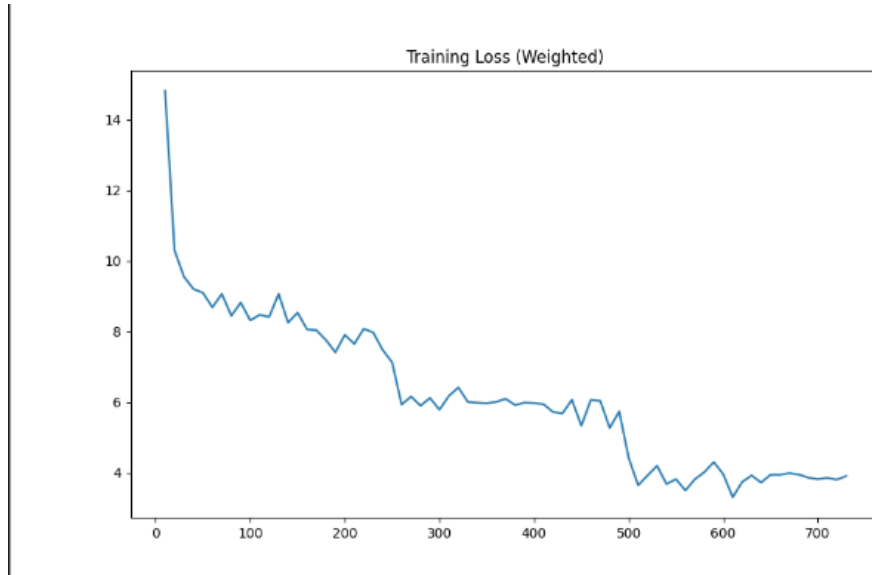


Figura 3.1: Training Loss (Weighted)

3.2.3 Esperimento 3: Training su Dataset Aumentato

La configurazione basata sul dataset aumentato ha restituito le performance complessive migliori, validando pienamente l'efficacia della strategia di *Data Augmentation* condotta tramite la generazione sintetica con il modello Zephyr.

Grazie al bilanciamento delle classi e all'introduzione di scenari d'attacco moderni, il modello ha dimostrato di aver appreso non solo la sintassi corretta delle comunicazioni aziendali, ma anche le più sottili sfumature stilistiche che differenziano una mail legittima da un tentativo di frode.

Tuttavia, nonostante la pulizia preliminare effettuata sui dati di training, l'analisi qualitativa degli output in fase di test ha evidenziato la persistenza di alcune imperfezioni, riconducibili alla natura intrinseca dei Large Language Models.

- In prima istanza, è stato riscontrato un fenomeno di *prompt leaking*, ovvero la presenza residua di frammenti delle istruzioni originali o delle risposte conversazionali del modello "insegnante" all'interno del testo generato, che ne comprometteva la verosimiglianza immediata.
- Oltre a questi residui testuali, sono emerse occasionali instabilità strutturali nel processo di generazione.
- In una buona percentuale di casi, il modello ha prodotto output nulli relativi all'oggetto e/o al body dell'e-mail, indicando che alcuni campioni che presentano tale caratteristica sono ancora presenti nel dataset.
- In altri frangenti, si è manifestato il fenomeno del *looping*, dove il sistema entrava in un ciclo ripetitivo, duplicando la stessa frase o i saluti finali all'infinito senza riuscire a concludere il messaggio in modo organico.
- Infine, sono stati rilevati segnali di *overfitting* stilistico, con il modello che tendeva a riutilizzare con eccessiva frequenza specifici nomi propri, aziende fittizie o domini presenti nel dataset di addestramento, riducendo la varietà percepita degli scenari.

La presenza di queste anomalie non ha indicato un fallimento della fase di addestramento, ma ha reso evidente la necessità di implementare un sistema di pulizia dell'e-mail generata. Nel capitolo successivo, verrà illustrato nel dettaglio come queste problematiche siano state gestite e risolte dinamicamente attraverso un modulo di *Post-Processing*.

Analisi della Loss: Come osservabile nel grafico comparativo 3.2, l'andamento dell'addestramento è nettamente superiore rispetto alle prove precedenti:

- **Convergenza Rapida:** La curva scende stabilmente sotto 1.5, evidenziando un apprendimento fluido e privo di incertezze.
- **Minimo Globale:** La loss finale attorno a 1.0 indica un'elevata capacità predittiva e una generazione coerente.
- **Effetto Scheduler:** Il netto calo allo step 500 conferma l'efficacia della riduzione del Learning Rate nel raffinare ulteriormente il modello.

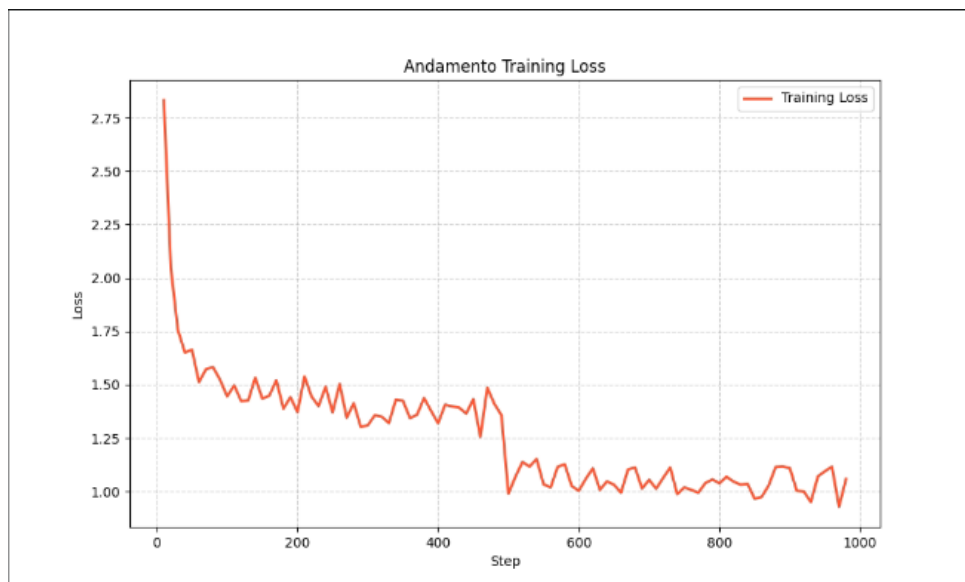


Figura 3.2: Andamento Training Loss (Dataset Aumentato)

Capitolo 4

Backend – Generazione e-mail e logica di gioco

Il backend rappresenta il cuore del sistema, orchestrando le due fasi critiche del gameplay: la generazione dinamica dei contenuti e la gestione della logica di gioco. Di seguito viene analizzata l'implementazione tecnica di questi componenti.

4.1 Generazione Mail

Per la fase di generazione, il sistema integra il modello Mistral-7B precedentemente sottoposto a *Fine-Tuning* sulla Versione 3 del dataset.

Nonostante l'addestramento specifico, durante i test è emerso che il modello tendeva a convergere verso pattern standardizzati se non adeguatamente stimolato. Il *fine-tuning*, infatti, era stato ottimizzato per l'apprendimento della struttura sintattica e semantica (distinzione *Legit* vs *Phishing*), ma non includeva una caratterizzazione in termini di "Role-Playing" o livelli di complessità variabile.

Per ovviare a questa limitazione e garantire una progressione della difficoltà nel gioco, è stata implementata una strategia di Prompt Engineering dinamico. Al momento della generazione, il backend inietta nel modello un *System Prompt* specifico che varia in base al livello di competenza raggiunto dall'utente (quest'ultimo gestito dal modulo di Reinforcement Learning, che verrà approfondito in seguito):

- **Livello Phishing Base:** Vengono richiesti attacchi di phishing espliciti e grammaticalmente imperfetti (es. "Urgent Password Reset"), facilmente riconoscibili.
- **Livello Phishing Intermedio:** Il prompt richiede scenari più formali, come comunicazioni HR o notifiche di servizi cloud, con una sintassi corretta.
- **Livello Phishing Avanzato:** Solo per giocatori che hanno raggiunto un livello di skill elevato, vengono sollecitati attacchi di *Spear Phishing* sofisticati, come risposte a thread di e-mail precedenti o fatture false contestualizzate, privi di errori evidenti e basati sulla manipolazione psicologica sottile.
- **Livello Legit E-mail:** Con probabilità casuale, l'IA genera un'email legit che si alterna a quella phishing corrispondente al livello utente.

Questa architettura permette di utilizzare un unico modello addestrato per coprire l'intero spettro di difficoltà del gioco.

4.2 Post-Processing e Sanitizzazione delle E-mail

Come evidenziato nell'analisi dei risultati (Capitolo 3), l'output grezzo del modello LLM può presentare anomalie strutturali o residui indesiderati. La presenza di questi artefatti non invalida la qualità del modello, ma necessita di un livello intermedio di controllo prima che il contenuto venga mostrato al giocatore.

Per garantire un'esperienza utente fluida e professionale, è stata sviluppata una pipeline di Post-Processing basata su Espressioni Regolari (*Regex*) e logiche euristiche in Python. Le funzioni di pulizia implementate intervengono sui seguenti aspetti:

- **Rimozione Artifacts e Meta-testo:** Vengono eliminati automaticamente prefissi generati dal modello in risposta alla richiesta o residui di prompt che potrebbero svelare la natura artificiale del messaggio.
- **Troncamento dei Loop:** Un algoritmo specifico rileva e interrompe le sequenze di testo ripetitive (es. firme duplicate o frasi ridondanti), tagliando l'email alla sua conclusione logica naturale.
- **Sanitizzazione PII (Personally Identifiable Information):** Per evitare bias cognitivi dovuti alla memorizzazione di nomi specifici (overfitting), una funzione di sostituzione rimpiazza dinamicamente i nomi propri, le aziende e i domini ricorrenti con placeholder generici o variabili randomizzate, aumentando la variabilità percepita.
- **Gestione degli Errori Critici (NaN/Empty):** Nel caso in cui il modello generi un output vuoto o non valido, il sistema intercetta l'errore e innesca automaticamente una ri-generazione silenziosa (*retry mechanism*), rendendo il problema trasparente all'utente finale.

4.3 Reinforcement Learning

Il modulo di Reinforcement Learning rappresenta una componente fondamentale dell'architettura del *Serious Game*, in quanto gestisce la logica di gioco e, nello specifico, l'adattamento della difficoltà. L'obiettivo è mantenere coerenti le sfide proposte con le competenze dimostrate dal giocatore.

4.3.1 Versione scartata - Approximate Policy Evaluation

Nelle fasi iniziali di progettazione, è stato esplorato un approccio basato su *Approximate Policy Evaluation*. Questa versione fondava il proprio funzionamento su un'inizializzazione dello stato dell'agente strettamente dipendente dal profilo di difficoltà selezionato dall'utente.

Nello specifico, il sistema assegnava un coefficiente numerico alla variabile `skill.user` (fissato, ad esempio, a 0.2 per un profilo *Junior* e a 0.7 per un *Senior*), stabilendo così il punto di partenza della curva di difficoltà.

A supporto di questo modello decisionale, era stata definita una **matrice dei pesi** di dimensioni 3x3. Questa struttura dati aveva il compito di mappare le tre possibili azioni dell'agente (generazione di email *Legit*, *Phishing Easy* o *Phishing Hard*) rispetto alle tre

features che costituivano lo stato: il termine di *Bias*, la *Skill* stimata dell'utente e il livello di *Difficoltà* intrinseca.

La prima e-mail generata, tuttavia, non seguiva la logica decisionale stocastica dell'agente basata su tale matrice ma veniva adottata una strategia in cui il sistema forzava deterministicamente la generazione di uno scenario coerente con il profilo scelto (ad esempio, un'email di tipo *Phishing Easy* per l'utente Junior o *Phishing Hard* per il Senior).

A conclusione di questo primo turno, ipotizzando un esito positivo per l'utente, il sistema calcolava un *Reward* proporzionale alla difficoltà superata (es. +5 per scenari Easy, +20 per Hard).

Questo feedback innescava la prima fase di apprendimento, stimolando l'aggiornamento dei pesi: la matrice dei pesi veniva aggiornata tramite un algoritmo di discesa del gradiente. Tale aggiornamento era selettivo, modificando esclusivamente la riga dei pesi corrispondente all'azione appena compiuta, con l'obiettivo di minimizzare l'errore di predizione tra il reward atteso e quello effettivamente ottenuto.

Dal secondo turno in poi, l'agente passava a una modalità decisionale autonoma. Per determinare la generazione successiva, il sistema calcolava la *Action-Value Function* $Q(s, a)$ per ogni possibile azione, ottenendo il valore Q tramite un'approssimazione lineare data dal prodotto tra le feature correnti e i pesi appresi.

La selezione finale dell'azione era infine governata da una policy epsilon-greedy, introdotta per il principio di Exploitation-Exploration.

Nonostante l'apparente validità teorica, l'idea è stata scartata perché nei test la difficoltà non si adattava mai all'utente. Il motivo potrebbe essere che l'algoritmo era troppo lento ad apprendere ed aveva bisogno di molte partite per calibrarsi correttamente (pesi della matrice). Per il nostro Serious Game, le sessioni di gioco durano pochi minuti e di conseguenza, l'IA non faceva in tempo a comprendere il giocatore, rendendo l'adattamento della difficoltà praticamente inesistente.

4.3.2 Versione implementata – Algoritmo ELO-like

In questa versione, lo stato del sistema è rappresentato univocamente dalla variabile *skill_utente* (un valore tra 0.0 e 1.0) e la selezione della difficoltà dell'e-mail da generare non è delegata a una scelta dell'IA, bensì a una logica a soglie statiche: se la skill è bassa (< 0.45) il sistema genera *Phishing Easy*, se supera una certa soglia passa a *Hard*, se oltre a 0.8 ad *Hard+*. In maniera dinamica e stocastica, la probabilità di generare un'e-mail legittima è legata alla competenza del giocatore: parte da una base fissa del 10% a cui si somma un ulteriore 15% proporzionale al livello di skill attuale. Se un valore casuale generato dal sistema rientra in questa soglia, viene generata un'email Legit.

$$\text{Chance of Legit} = 0.1 + (\text{Skill} \times 0.15)$$

Il livello di skill utente iniziale è impostato a 0.25 per il profilo junior e 0.75 per il senior.

L'elemento chiave di questa versione è il **sistema di reward a guadagno variabile**. L'aggiornamento della competenza non avviene tramite un punteggio fisso, ma segue una formula che modula l'incremento in base al livello attuale dell'utente:

- **In caso di Vittoria:** L'aumento della skill è inversamente proporzionale alla skill stessa. La presenza del learning rate impostato a 0.12 per il livello junior e a 0.05 per il livello senior è il meccanismo che definisce la velocità di apprendimento e progressione. Un learning rate più alto per i principianti garantisce un avanzamento rapido

nelle fasi iniziali, mentre un valore più basso per gli esperti rende l'incremento della skill più severo e lento, richiedendo una buona costanza nel dare le risposte corrette per raggiungere i vertici della difficoltà, la fine della partita. Questo meccanismo serve a stabilizzare la curva di apprendimento ai livelli alti, impedendo che pochi colpi di fortuna portino il giocatore subito al livello massimo e assicurando che la difficoltà *Hard+* venga raggiunta solo con una reale padronanza. In pratica, più un utente è bravo, meno viene premiato e più risposte corrette dovrà fornire per aumentare significativamente il suo livello di skill:

$$\text{Guadagno Base} = \text{Learning Rate} \times (1.05 - \text{Skill Attuale})$$

Questa formula applica il principio matematico del rendimento decrescente. Il termine **Learning Rate** ha la funzione di moltiplicatore base che stabilisce la velocità generale di progressione in base al profilo scelto. Il fattore **(1.05 - Skill Attuale)** fa in modo che, quando la competenza dell'utente è bassa, il moltiplicatore sia grande, garantendo un punteggio elevato. Man mano che la skill cresce, la differenza si assottiglia, diminuendo il valore del termine e riducendo i punti base guadagnati.

In fase di testing, è stata confermata la bontà di questo sistema di reward per quanto riguarda il concetto di sfida progressivamente più difficile, ma, nella pratica, risultava frustrante ottenere premi sempre più bassi e dover giocare un gran numero di round prima di arrivare alla fine della partita. Per bilanciare questo aspetto e mantenere alta la motivazione, è stato integrato un sistema di valutazione basato sul modello addestrato. Dopo ogni scelta, l'utente è obbligato a inserire una motivazione testuale a supporto della risposta data, che viene valutata dall'IA e alla quale viene assegnato una *score* da 0 a 10. A seconda di questo voto, il Guadagno Base viene rimodulato e viene applicato un Bonus Ragionamento:

- **Fascia Alta (Voto 7-10):** La motivazione è corretta e logica. L'utente ottiene il 100% del Guadagno Base calcolato dalla formula, più un **Bonus Massimo fisso (+0.04)**.
- **Fascia Media (Voto 4-6):** La motivazione è buona ma presenta inesattezze o piccole allucinazioni (es. confusione tra brand o citazione di allegati inesistenti). L'utente ottiene solo il 50% del Guadagno Base, più un **Bonus Minimo fisso (+0.02)**.
- **Fascia Bassa (Voto 1-3):** La risposta è una palese contraddizione logica (es: l'utente ha cliccato phishing ma ha detto che l'email è legittima). Il guadagno non viene assegnato.
- **Fascia "Zero" (Voto 0):** I casi in cui la risposta non è stata inserita, è casuale, è una resa o presenta parole a caso. Il sistema blocca l'utente e chiede di reinserire la motivazione, invitando l'utente a ragionare.

Questo bonus fisso garantisce che, anche ai livelli più alti dove la formula matematica abbatta il guadagno base, il giocatore continui a percepire un progresso tangibile a patto di dimostrare un'effettiva comprensione della minaccia.**

Per realizzare quanto detto, al modello sono stati dati degli esempi mirati a coprire diversi scenari di email e motivazioni possibili, usando una strategia di prompt engineering.

- **In caso di Sconfitta:** Il sistema applica una penalità che riduce la skill, facendo retrocedere l'utente verso soglie di difficoltà inferiori. Questo permette al giocatore di recuperare e consolidare le basi su esempi più semplici prima di affrontare nuovamente sfide complesse.

Gestione Partita

La partita è strutturata come una sessione di addestramento a tentativi limitati, in cui l'obiettivo del giocatore è raggiungere la padronanza completa nel riconoscimento delle minacce informatiche prima di esaurire le possibilità di errore a sua disposizione.

Il sistema è gestito da un contatore di "vite", il cui ammontare iniziale dipende dal profilo di difficoltà selezionato in fase di avvio. Nello specifico:

- Al profilo **Junior** vengono assegnate **4 vite**, garantendo un margine di tolleranza più ampio per favorire l'apprendimento esplorativo e assorbire gli errori iniziali.
- Al profilo **Senior** vengono concesse solamente **2 vite**, rendendo l'esperienza nettamente più difficile e stimolante per un utente esperto.

Durante il normale ciclo di gioco, ogni classificazione errata di un'e-mail comporta la decurtazione di una vita. Se il contatore delle vite raggiunge lo zero, la partita si interrompe decretando la sconfitta dell'utente (*Game Over*).

Al contrario, la condizione di **vittoria** si innesca quando il giocatore, riuscendo a fornire risposte corrette e supportate da buone/ottime motivazioni logiche valutate dall'IA, riesce a incrementare la propria competenza fino a far sì che la *skill* utente raggiunga o superi il valore soglia di **1.0**. Il superamento di questa soglia certifica l'avvenuto apprendimento dei concetti di sicurezza, interrompendo la partita e attestando che l'utente è in grado di riconoscere sistematicamente anche gli attacchi più sofisticati.

Capitolo 5

Motivator

Oltre alla generazione dinamica degli scenari, un componente essenziale del *Serious Game* è la capacità di fornire un feedback educativo immediato. Non è sufficiente comunicare all’utente se ha “Vinto” o “Perso”; per garantire un apprendimento efficace, è necessario spiegare il perché e quali caratteristiche è bene individuare.

A tale scopo è stato sviluppato un **Motivator**, un sistema basato su Intelligenza Artificiale progettato per analizzare l’e-mail appena “giocata” e generare una spiegazione in linguaggio naturale che evidenzia gli indizi di malevolenza (nel caso di phishing) o gli elementi di sicurezza (nel caso di e-mail legittime).

Anche nello sviluppo di questa parte del progetto, sono stati effettuati diversi test che hanno compreso due approcci principali.

5.1 Motivator Mistral

Nel primo esperimento, si è tentato di riutilizzare il modello *Mistral-7B*, già impiegato per la generazione delle e-mail, sfruttando la sua capacità di ragionamento generalista. In questa configurazione, al termine del turno di gioco, il sistema inviava al modello il testo dell’e-mail appena visualizzata corredato da un prompt specifico che richiedeva di spiegarne la natura.

Tuttavia, i test hanno evidenziato due criticità bloccanti che hanno portato a scartare questa soluzione:

- **Qualità del Feedback:** Le risposte ottenute, seppur semanticamente corrette, risultavano eccessivamente lunghe e generiche. Il modello tendeva a fornire consigli di sicurezza standard piuttosto che analizzare lo specifico campione in esame.
- **Onerosità Computazionale:** L’utilizzo di un modello da 7 miliardi di parametri per un task di classificazione e spiegazione ha introdotto una latenza non accettabile per il nostro gioco: se il tempo di attesa per la generazione di ogni email è accettabile e giustificato/mitigato dal tempo che l’utente impiega per leggere, ragionare e rispondere, questo non vale per la generazione del consiglio che dovrebbe essere immediato.

5.2 Motivator Fine Tuning – TinyLlama

Per superare i limiti di latenza e genericità, la scelta definitiva è ricaduta su **TinyLlama-1.1B**, un modello estremamente compatto con 1.1 miliardi di parametri.

A differenza dell'approccio precedente, il modello non è stato utilizzato in modalità *Zero-Shot*, ma è stato sottoposto a un processo di *Supervised Fine-Tuning* mirato. È stato creato un dataset sintetico specifico composto da coppie [Input: Testo Email] -> [Output: Spiegazione Didattica].

Il dataset è stato creato raccogliendo un campione di 200 e-mail generate sinteticamente sfruttando il modello alla base del backend e, successivamente, sono state generate sinteticamente, tramite Gemini, le istruzioni per identificare pattern specifici come *Typosquatting*, *Urgency*, *Sensitive Data Request* o *Mismatch Sender*.

Risultati ottenuti: L'addestramento specifico ha trasformato TinyLlama in un analista esperto, offrendo vantaggi decisivi:

- **Analisi Contestuale:** Il modello ha imparato a individuare sull'indizio esatto presente nel testo evitando generalismi.
- **Efficienza Estrema:** Grazie alle dimensioni ridotte dell'architettura, il feedback appare istantaneamente a schermo, mantenendo alto il ritmo di gioco e garantendo un rinforzo positivo immediato per l'apprendimento.

Tra le note negative, abbiamo constatato che alcuni feedback non corrispondono precisamente al tipo di e-mail effettivo, ma questo problema è stato riscontrato un numero estremamente ridotto di volte. Inoltre, si è osservato che occasionalmente il *motivator* produce spiegazioni testuali contenenti errori sintattici. Questo fenomeno potrebbe essere riconducibile alla natura e all'architettura del modello utilizzato, infatti, TinyLlama è un *Large Language Model* compatto, con un numero di parametri nettamente inferiore rispetto al modello principale (Mistral) e ottimizzato per garantire tempi di risposta rapidi, sacrifica parte della sua fluidità linguistica.

Capitolo 6

Analisi delle Interazioni e Casi d'Uso

In questo capitolo vengono mostrati i diversi casi d'uso descritti nei capitoli precedenti, tradotti in scenari pratici di gioco. L'obiettivo è illustrare l'esperienza utente (*User Experience*) e il flusso delle interazioni attraverso una serie di schermate catturate durante sessioni reali di utilizzo del *Serious Game*, che ne illustrano le features principali.

6.1 Avvio del Gioco e Scelta del Profilo

All'avvio dell'applicativo, l'utente viene accolto dalla schermata principale, la quale ha il compito di introdurre l'obiettivo didattico e richiedere la selezione del livello di competenza iniziale. La scelta del profilo (Junior o Senior) ha un impatto immediato e visibile sull'interfaccia utente. Le figure 6.1 e 6.2 mostrano il menù iniziale e le possibilità di scelta del profilo iniziale:

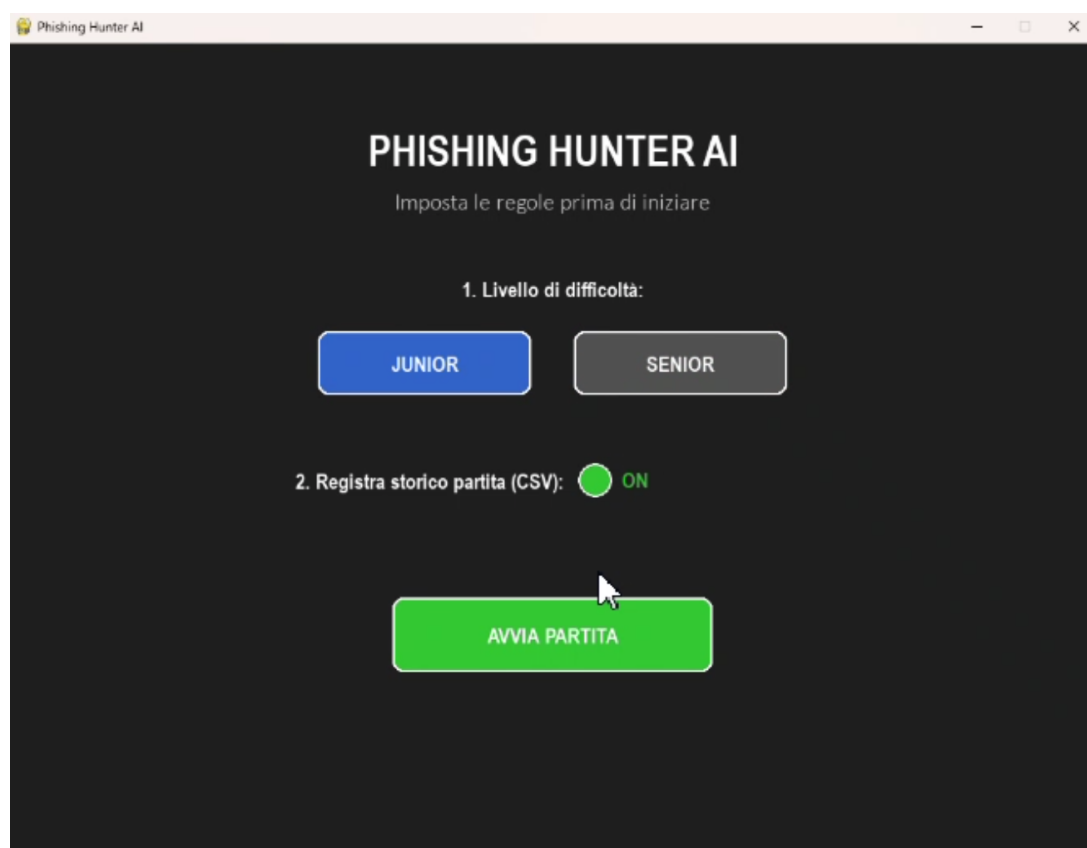


Figura 6.1: Schermata di avvio e utente che sceglie il profilo Junior

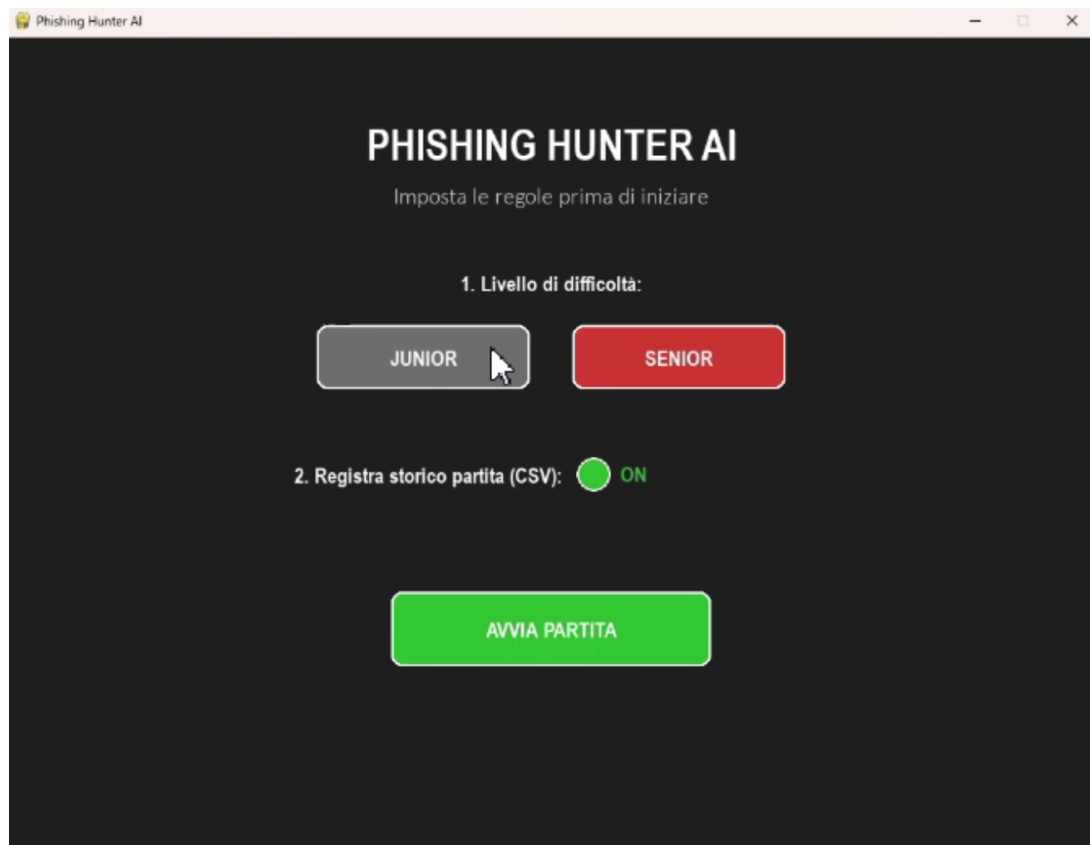


Figura 6.2: Schermata di avvio e utente che sceglie il profilo Senior

6.2 Analisi degli Scenari

Una volta iniziata la partita, il flusso del gioco si struttura attorno alla lettura e all'analisi delle e-mail. L'interfaccia simula un tipico client di posta elettronica aziendale. Il testo generato dinamicamente dal modello LLM viene presentato all'utente completo di Oggetto e Corpo del messaggio. Nelle Figure 6.3, 6.4, 6.5 e 6.6 è possibile osservare esempi di email di phishing di diversa difficoltà, un esempio di un'email legittima e notare il conteggio delle vite in base al profilo scelto.

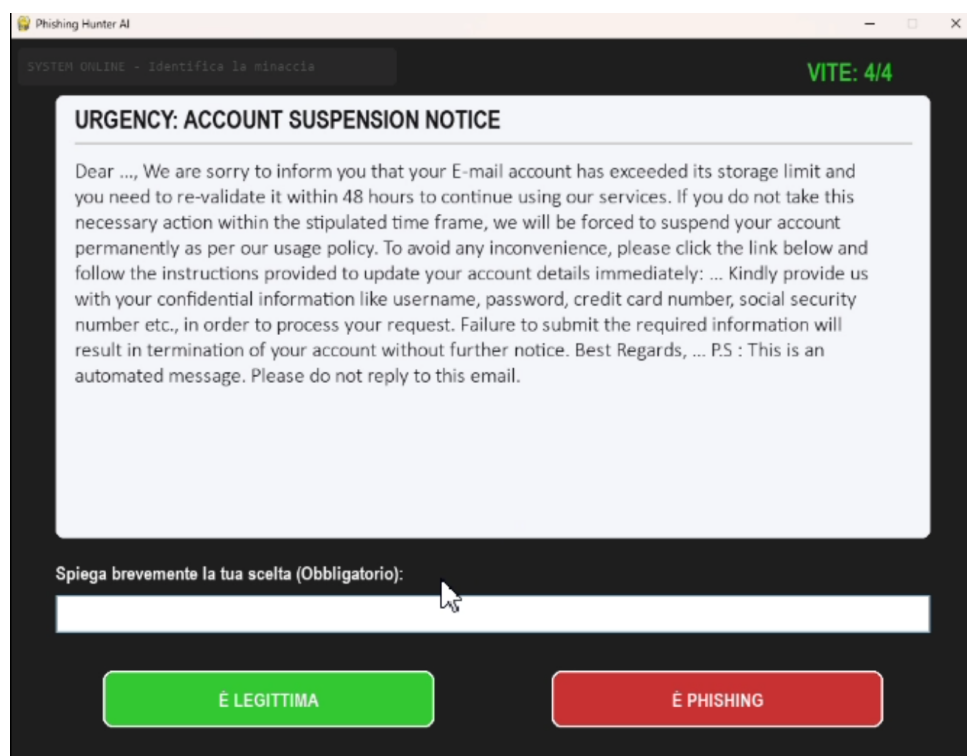


Figura 6.3: Esempio di visualizzazione di un'e-mail di Phishing Easy generata dinamicamente con profilo Junior selezionato.

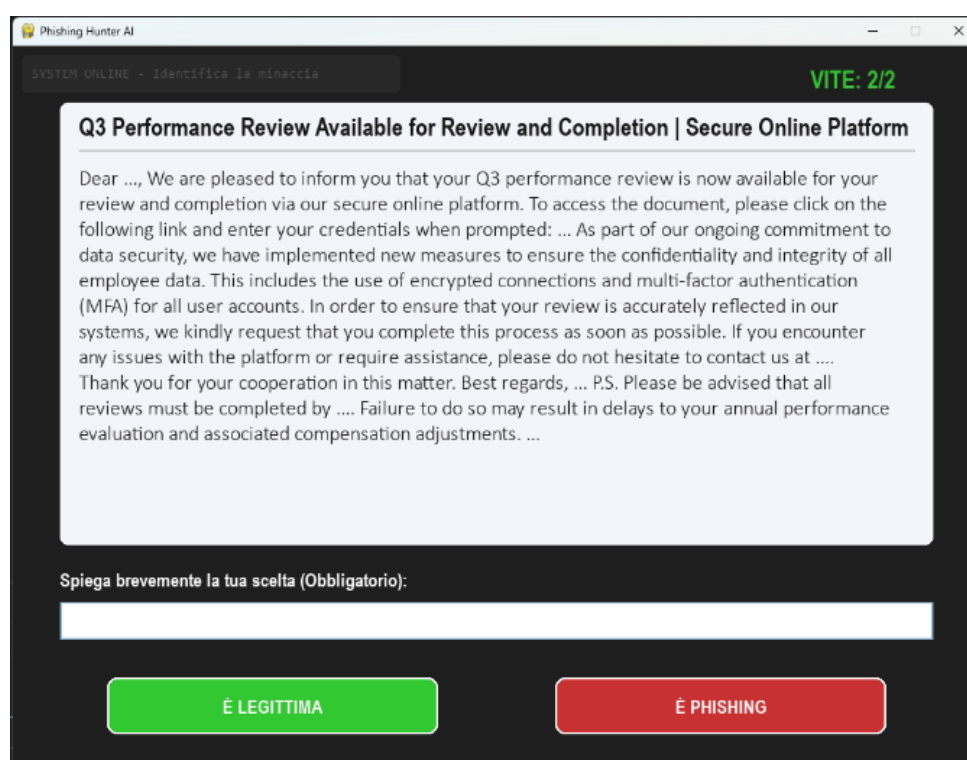


Figura 6.4: Esempio di visualizzazione di un'e-mail di Phishing Hard generata dinamicamente con profilo Senior selezionato.

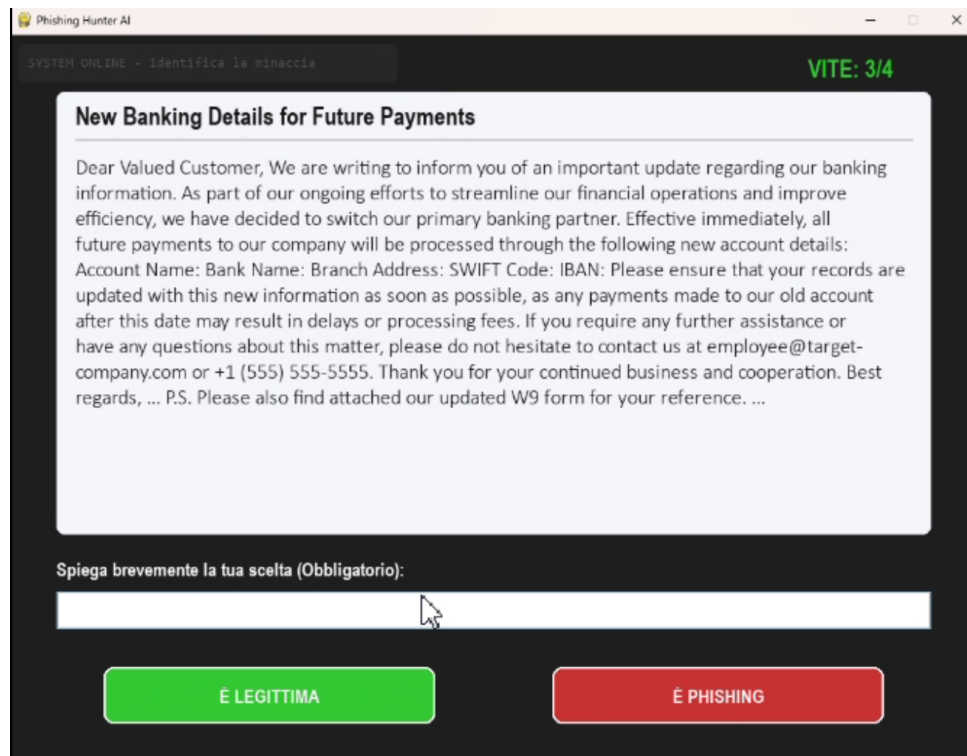


Figura 6.5: Esempio di visualizzazione di un'e-mail di Phishing Hard+.

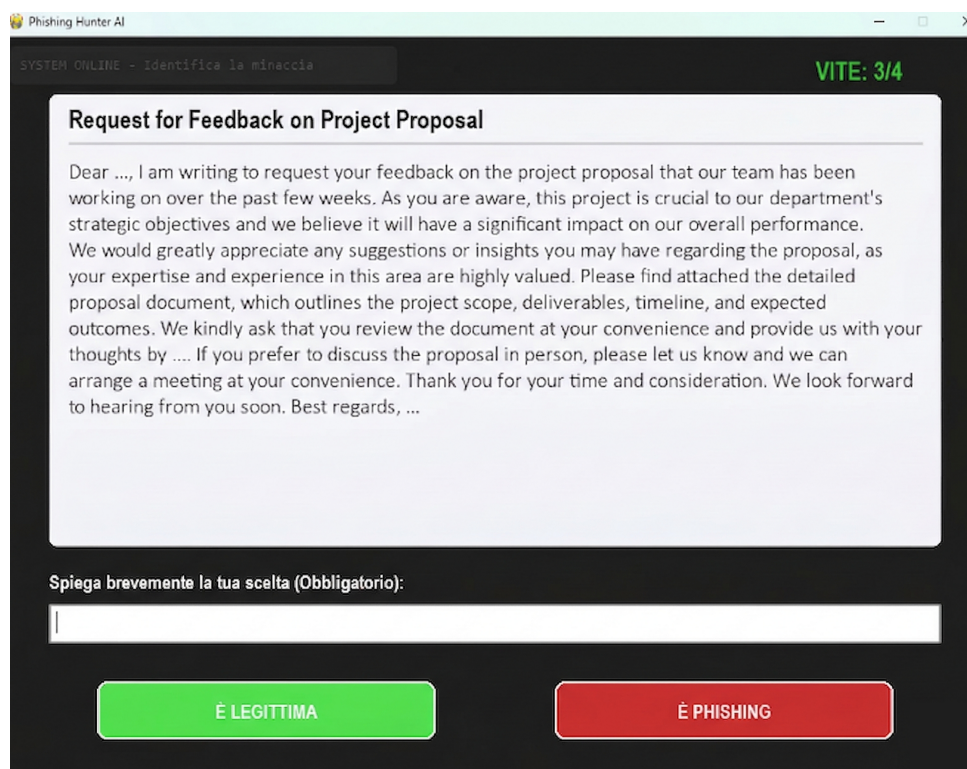


Figura 6.6: Esempio di visualizzazione di un'e-mail Legit.

6.3 Inserimento della Motivazione e Valutazione

Il sistema richiede l'intervento attivo del giocatore tramite un'apposita casella di testo, obbligandolo a fornire una giustificazione logica per la sua scelta, che accompagna la selezione. Questo passaggio è cruciale per attivare il modulo *Motivator*. In caso di risposta corretta e ben argomentata (Figura 6.7), l'IA assegna un punteggio elevato, eroga il *Bonus Ragionamento* massimo e restituisce un feedback testuale che contestualizza la minaccia. Viceversa, qualora l'utente fornisca spiegazioni scollegate dal testo, l'IA penalizza severamente la valutazione, annullando il guadagno di *skill* e invitando l'utente a un'analisi più attenta (Figura 6.9).

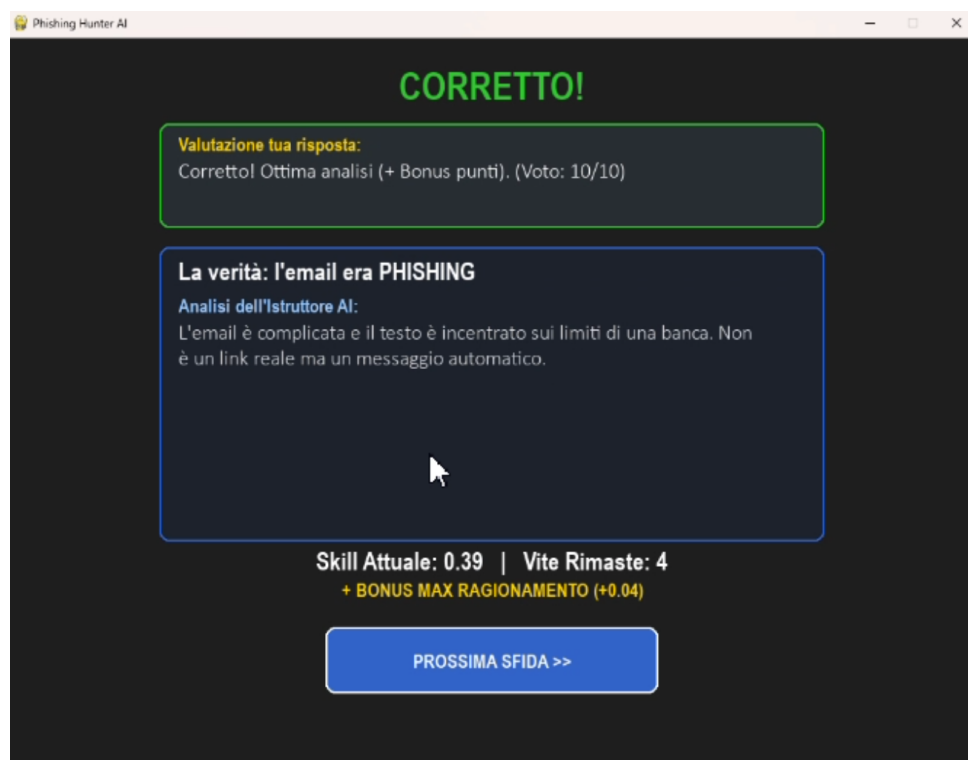


Figura 6.7: Feedback positivo del Motivator: punteggio elevato e bonus alto applicato.

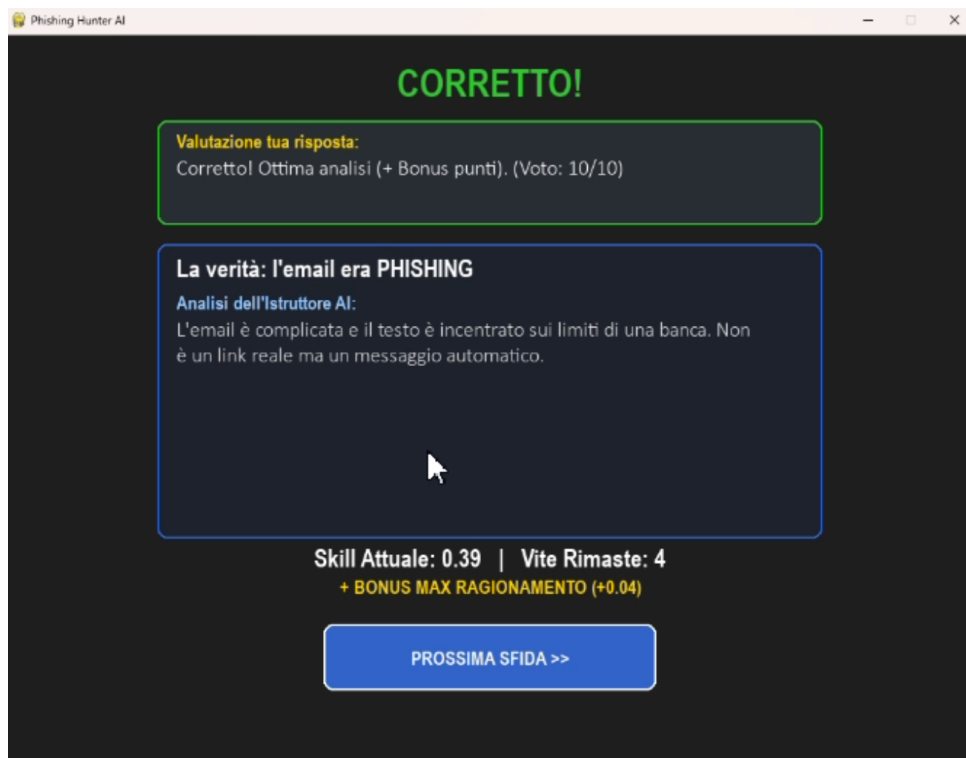


Figura 6.8: Feedback positivo del Motivator: punteggio medio e bonus basso applicato.

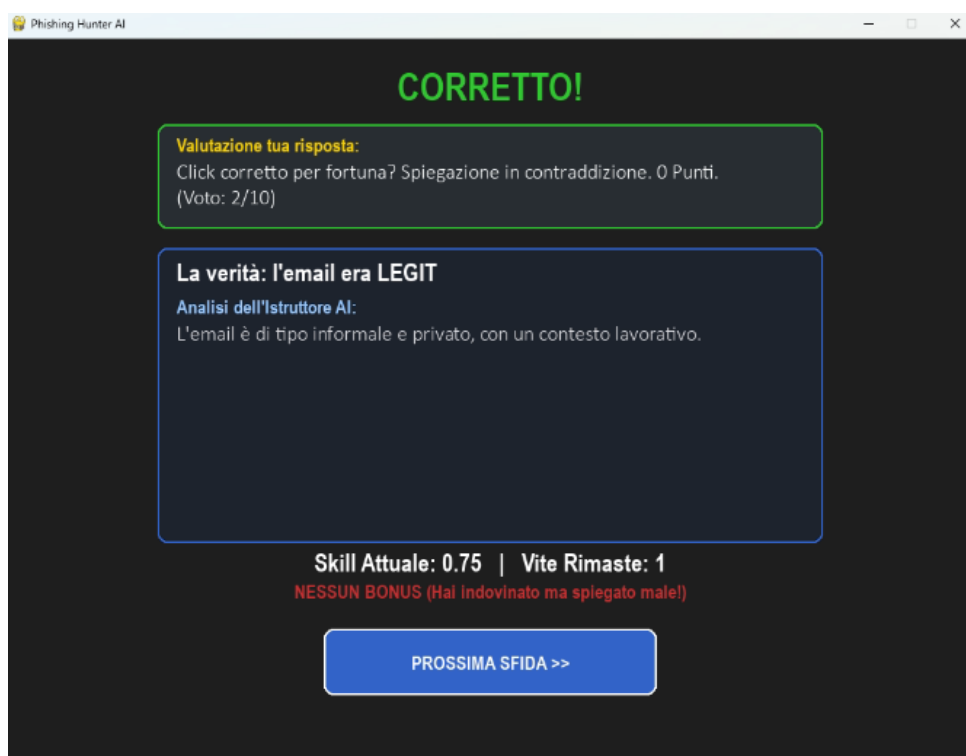


Figura 6.9: Feedback negativo del Motivator a seguito di una motivazione errata.

6.4 Condizioni di Fine Partita (Win & Game Over)

L'esperienza di gioco termina al verificarsi di una delle due condizioni previste dal sistema. Il fallimento (*Game Over*) subentra qualora il giocatore esaurisca le vite, venendo accolto da una schermata riepilogativa che ne decreta la sconfitta (Figura 6.10). Il successo (*Win*) viene invece raggiunto quando la concatenazione di scelte corrette e motivazioni logiche valide porta la *Skill* dell'utente a superare la soglia massima di 1.0. In questo scenario (Figura 6.11), l'interfaccia restituisce un messaggio di congratulazioni, certificando la completa padronanza dell'utente nel riconoscimento delle minacce informatiche.

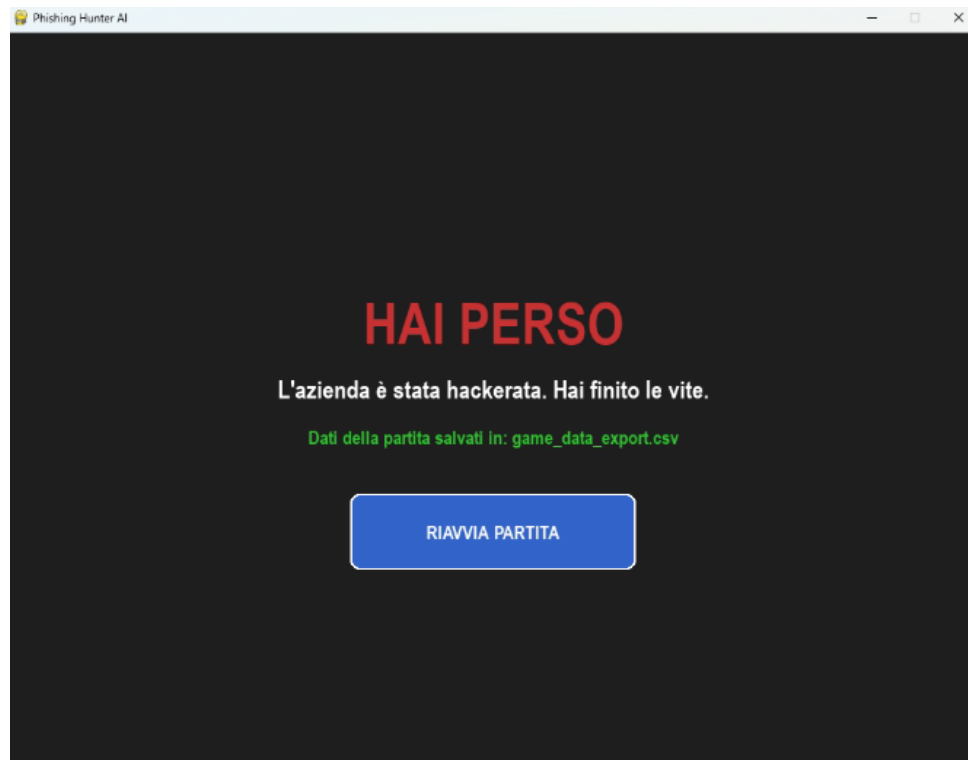


Figura 6.10: Schermata di Game Over per esaurimento dei tentativi disponibili.

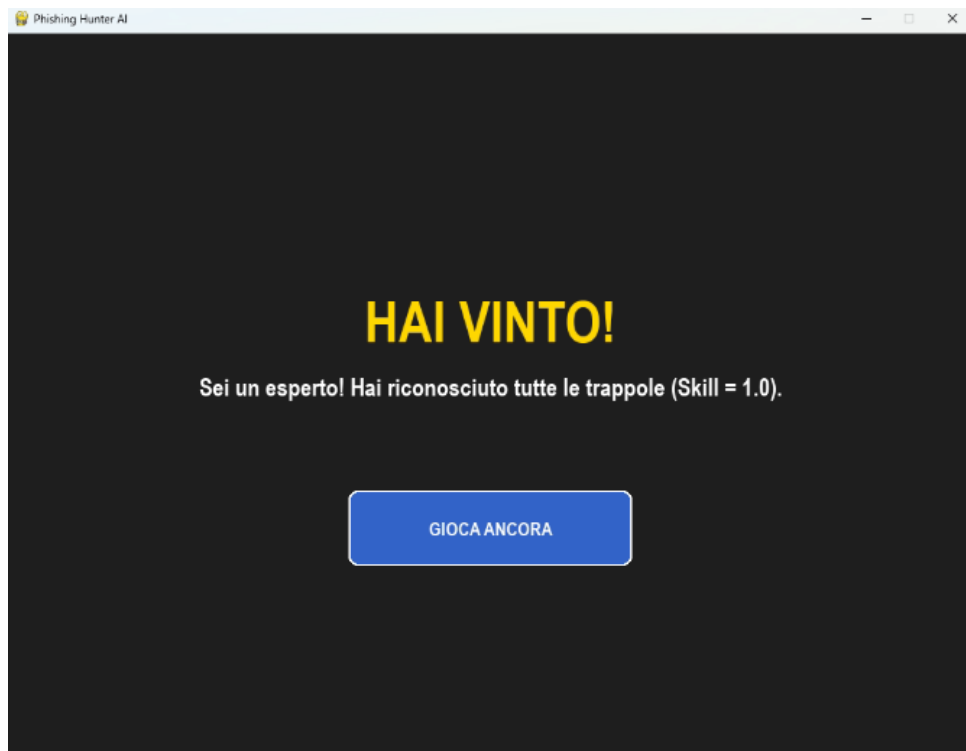


Figura 6.11: Schermata di Vittoria al raggiungimento della skill massima (1.0).

Capitolo 7

Conclusioni e Sviluppi Futuri

Il progetto *PhishingHunterAI* ha dimostrato la fattibilità e l'efficacia dell'integrazione di *Large Language Models* all'interno di un *Serious Game* adattivo, superando i limiti di staticità e ripetitività tipici dei tradizionali simulatori di phishing. L'architettura proposta, che combina la generazione degli scenari con un algoritmo di *Reinforcement Learning* ELO-like per il bilanciamento dinamico della difficoltà, è riuscita a creare un ambiente di apprendimento in grado di mantenere alto il livello di sfida dell'utente. Il modulo *Motivator* ha inoltre confermato l'importanza di un feedback didattico contestuale e immediato.

Tuttavia, lo sviluppo e la sperimentazione hanno portato alla luce alcune criticità, aprendo la strada a diverse prospettive di miglioramento per le versioni future del sistema.

7.1 Limiti Riscontrati

Le principali limitazioni attuali si concentrano sulle componenti generative dell'architettura:

- **Artefatti e Overfitting Stilistico:** Nonostante il *Data Augmentation* e le procedure di *Post-Processing*, il modello generatore (Mistral) mostra occasionalmente fenomeni di *prompt leaking* o tende a riutilizzare pattern sintattici e nomi fittizi troppo frequenti nel dataset di training, riducendo l'imprevedibilità a lungo termine.
- **Precisione Linguistica del Motivator:** Come analizzato nel Capitolo 5, l'utilizzo di un modello estremamente compatto come TinyLlama (1.1B parametri) garantisce una risposta semplice e immediata, ma fornisce talvolta spiegazioni grammaticalmente imperfette.

7.2 Prospettive di Miglioramento

Per superare i limiti esposti e raffinare ulteriormente le dinamiche di gioco, si propone di utilizzare un dataset ulteriormente ricco di esempi, pulito e eterogeneo generato sinteticamente per fare il fine tuning del modello per la generazione di email e del feedback, con lo stesso approccio, creare dataset anche per il modulo della valutazione della risposta utente al posto di utilizzare la strategia di Few Shot Prompting. Per rendere il sistema ancora più intelligente, considerare più di una metrica per valutare il livello di bravura dell'utente in base all'abilità mostrata nel rispondere ad email di uno scenario/tema specifico.

In conclusione, *PhishingHunterAI* mostra che l'AI può essere usata per creare Serious Game didattici utili in ambito scolastico e lavorativo.